DATABASE MANAGEMENT SYSTEMS

FIVE ANSWERS — 20 MARKS EACH

Name: Mohamed Farhan
Register Number: 1925260188
Course Code: CSA05
Assessment: Continuous Internal Assessment — Concept Mapping

Prepared in black-and-white format with explanations and concept maps.

QUESTION 1 — EER MODEL: GENERALIZATION, SPECIALIZATION AND AGGREGATION

Introduction: The Enhanced Entity–Relationship (EER) model extends the basic ER model by representing more complex real-world situations. It supports concepts such as superclasses and subclasses, inheritance, specialization, generalization and aggregation. In a university database, Person can be a general entity, while Student, Faculty and Staff are specialized entities.

1. Basic ER model and EER model

The ER model describes data through entities, attributes and relationships. An entity is an identifiable object, an attribute describes it, and a relationship connects entities. The EER model retains these features and adds semantic concepts that make complex designs clearer.

2. Generalization

Generalization is a bottom-up process. Two or more similar entity types are combined into a common superclass by identifying their shared attributes. For example, Student, Faculty and Staff may share PersonID, Name, DateOfBirth and Address. These common properties can be placed in the Person superclass.

3. Specialization

Specialization is a top-down process. A superclass is divided into subclasses based on distinguishing features. Student may have StudentID and Programme; Faculty may have FacultyID and Specialization; Staff may have StaffID and JobRole. Each subclass inherits the common attributes of Person and can have its own attributes.

Constraints: Disjoint specialization means an entity can belong to only one subclass in the specialization; overlapping specialization allows membership in more than one subclass. Total specialization means every superclass entity must belong to a subclass; partial specialization allows some superclass entities to belong to none of the subclasses.

4. Inheritance

Inheritance means subclasses receive the attributes and relationships of their superclass. Therefore, Student, Faculty and Staff inherit PersonID, Name, DateOfBirth and Address. This reduces repetition and supports consistent data design.

5. Aggregation

Aggregation treats a relationship set and its participating entities as a higher-level abstract unit. For example, a Faculty member teaches a Course, and a Student enrols in that Course. If the university needs to record that a Student evaluates a particular teaching assignment, the relationship Faculty–Teaches–Course can be treated as one unit and related to Evaluation. Aggregation is useful when a relationship itself must participate in another relationship.

Concept map

```
PERSON (Superclass)
|
+----------------+----------------+
| | |
STUDENT FACULTY STAFF
| | |
StudentID FacultyID StaffID
Programme Specialization JobRole
\ | /
+---- inherit common attributes ----+
PersonID, Name, DOB, Address

Generalization: Student + Faculty + Staff -> Person
Specialization: Person -> Student / Faculty / Staff
```

```
Aggregation: (Faculty -- Teaches -- Course) treated as a unit
that can participate in another relationship, e.g. Evaluation
```

6. Benefits and conclusion

EER modelling improves clarity, avoids repeated attributes, represents inheritance, and supports more accurate database design. Generalization combines similar entities, specialization separates entities by role, and aggregation models relationships that need to be treated as higher-level units. These concepts make the university database easier to understand and maintain.

Suggested 20-mark coverage: definitions and comparison (4), university example (5), concept map (5), constraints/inheritance/aggregation (4), benefits and clarity (2).

QUESTION 2 — DBMS ARCHITECTURE FOR AN ONLINE BANKING REQUEST

Scenario: A banking customer requests account information through an online banking application. The DBMS receives the query, checks it, coordinates safe execution, accesses stored data and returns the result.

1. Request flow

The user submits a request such as “display my account balance.” The application sends an SQL query to the DBMS. The DBMS processes the query, verifies permissions and coordinates the transaction before retrieving the relevant records.

Concept map / flow

```
CUSTOMER
|
v
ONLINE BANKING APPLICATION
| SQL request: SELECT balance FROM Account ...
v
QUERY PROCESSOR
|-- Parser and validator: checks SQL syntax and meaning
|-- Optimizer: selects an efficient execution plan
|-- Executor: runs the selected plan
v
TRANSACTION MANAGER
|-- Coordinates transaction execution
|-- Helps preserve ACID properties and concurrency control
v
STORAGE MANAGER
|-- File/index manager: locates records and pages
|-- Buffer manager: moves pages between disk and memory
|-- Authorization/integrity checks: enforces permitted access
v
DATABASE FILES / INDEXES / LOGS
| Read the required account record
v
RESULT RETURNS THROUGH DBMS
|
v
APPLICATION DISPLAYS ACCOUNT INFORMATION TO CUSTOMER
```

2. Query processor

The query processor handles the SQL statement. The parser checks syntax and validates referenced tables and columns. The optimizer compares possible execution plans and selects an efficient one. The executor carries out the plan and requests the necessary data from lower-level DBMS components.

3. Transaction manager

A banking request must be reliable because account data is sensitive. The transaction manager coordinates transactions and helps enforce ACID properties: Atomicity (all-or-nothing execution), Consistency (valid rules remain satisfied), Isolation (concurrent transactions do not interfere improperly) and Durability (committed changes survive failures). A read-only balance request may not modify the balance, but it still needs a consistent and authorized view of the data.

4. Storage manager and database files

The storage manager provides the interface between query execution and physical data. It manages files, indexes, memory buffers and disk access. Database files store account records; indexes can speed up searches by account number or customer ID; logs support recovery where applicable.

5. Retrieval and response

The DBMS locates the account record, retrieves the requested fields, and returns the result to the application. The application formats the result for the customer. Access controls must ensure that a user can view only authorized account information.

6. Importance

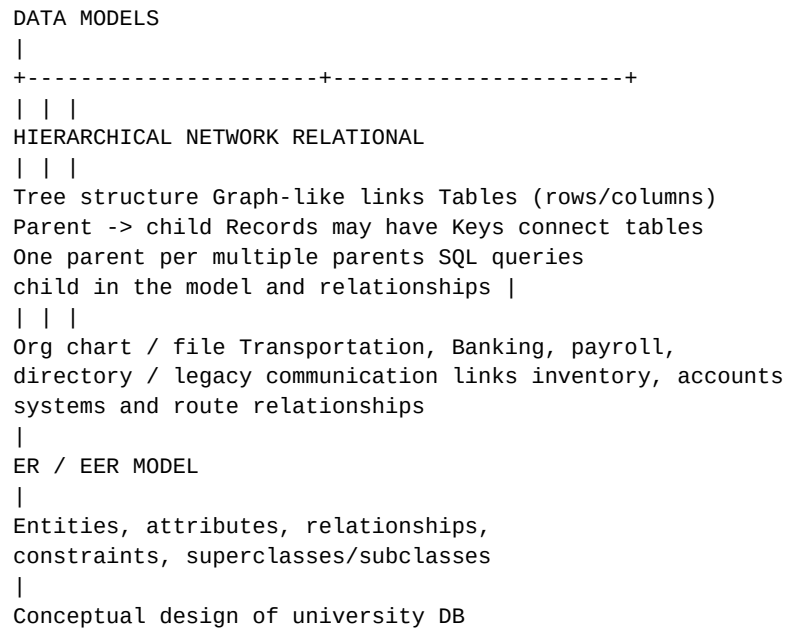
This architecture separates query interpretation, transaction coordination and physical data access. It improves maintainability, performance, security and reliability. In banking, correct authorization and transaction handling are particularly important.

Suggested 20-mark coverage: flow and map (6), query processor (4), transaction manager (4), storage/data files (4), security and conclusion (2).

QUESTION 3 — DATA MODELS AND THEIR REAL-WORLD APPLICATIONS

Introduction: A data model describes how data is organized, related, constrained and accessed. Different models are suitable for different kinds of relationships and application requirements.

Concept map



1. Hierarchical model

The hierarchical model organizes records as a tree with parent–child relationships. Each child typically has one parent, while a parent can have multiple children. It is suitable for naturally nested data, such as an organizational hierarchy, a folder directory or certain legacy systems. It can be efficient when access follows known paths, but it is less flexible for many-to-many relationships.

2. Network model

The network model organizes records as a graph of linked records. A record can be connected to multiple other records, supporting more complex relationships than a strict tree. It can represent transportation connections, route networks or communication links where locations may connect to many others. Its structure can be powerful but may be harder to design and navigate than relational tables.

3. Relational model

The relational model stores data in tables. Rows represent records and columns represent attributes. Primary keys identify rows, while foreign keys link related tables. SQL is used to insert, retrieve, update and delete data. It is suitable for banking because customers, accounts and transactions can be stored in related tables with integrity constraints. It is also widely useful for payroll, inventory and student records.

4. ER/EER model

The Entity–Relationship model is mainly used for conceptual database design. It represents entities, attributes and relationships before implementation in tables or another database structure. EER adds specialization, generalization, inheritance and aggregation. A university can model Person, Student, Faculty, Course and Enrolment, then translate the design into a relational schema.

5. Comparison

Hierarchical is best for tree-like parent–child data; Network supports richer linked-record relationships; Relational provides table-based structure, keys and declarative SQL; ER/EER communicates requirements and conceptual

relationships during design. The best choice depends on data complexity, access patterns, integrity requirements and maintainability.

Conclusion

Data models provide different ways to represent information. Hierarchical models fit tree structures, network models fit interconnected records, relational models fit structured business data, and ER/EER models are especially useful for designing a clear conceptual schema before implementation.

Suggested 20-mark coverage: definitions and concept map (5), four models and examples (10), comparison/suitability (3), conclusion (2).

QUESTION 4 — SCHEMA, INSTANCE AND DATA INDEPENDENCE

Introduction: A database has a design or structure and also contains actual data at a particular time. The schema describes the structure, while an instance is the current contents. The three-schema architecture separates user views, logical design and physical storage.

1. Schema and instance

A schema is the overall definition of a database structure, including tables, attributes, relationships and constraints. It changes relatively infrequently. An instance is the data stored in the database at a specific moment and changes whenever records are inserted, updated or deleted. For example, a Student table definition is part of the schema; the students currently listed in it form an instance.

2. External, logical and physical levels

The external level contains user-specific views of the database. The logical (conceptual) schema describes the overall logical structure, such as entities, tables, attributes and relationships, without specifying the physical storage details. The physical (internal) schema describes how data is stored, such as files, indexes, record layouts and storage organization.

Concept map

```
DATABASE SYSTEM
|
+--> SCHEMA = definition / structure
| |
| +--> Logical schema: tables, attributes,
| | relationships, constraints
| |
| +--> Physical schema: files, indexes,
| storage layout, access paths
|
+--> INSTANCE = actual data at a particular time

External views <--> Logical schema <--> Physical schema
| | |
| | +-- Physical data
| | independence
| +-- Logical data
| independence
+-- Different user views
```

3. Logical data independence

Logical data independence is the ability to change the conceptual or logical schema without requiring changes to external views or application programs, provided their required view remains supported. For example, adding a new attribute to a Student table or splitting a table into related tables may be possible without changing every user application if suitable views or mappings preserve the interface.

4. Physical data independence

Physical data independence is the ability to change the internal storage schema without changing the logical schema or application programs. For example, adding an index, changing file organization or moving database files to a different storage device should not require changes to the logical table definitions.

5. Difference

Logical data independence concerns changes at the logical level while protecting external views and programs. Physical data independence concerns changes at the physical level while protecting the logical schema. Physical changes are generally easier to isolate; logical changes can affect more user views and applications.

6. Why data independence matters

Data independence reduces maintenance costs, permits database performance tuning, supports system growth and protects applications from unnecessary changes. The schema-instance distinction also helps separate stable design from changing operational data.

Conclusion

Schema describes the database structure; instance describes its current data. Logical and physical schemas separate design from storage. Data independence allows changes at one level with minimal impact on the level above it.

Suggested 20-mark coverage: definitions (4), three levels and map (6), logical independence (4), physical independence (4), comparison/conclusion (2).

QUESTION 5 — DBA, END USER AND APPLICATION DEVELOPER

Introduction: A DBMS environment involves people with different responsibilities. The Database Administrator (DBA) manages and controls the database system, end users use the data to complete tasks, and application developers build software that accesses the database.

Concept map

```
ORGANIZATION / BUSINESS NEEDS
|
+------------+------------+
| | |
DBA APPLICATION END USER
DEVELOPER
| | |
Administers DB Builds UI, Enters, searches,
security, access, business logic updates and views
backup, recovery, and SQL/data permitted data
performance, access code
integrity |
| |
+---- DBMS / DATABASE ----+
^ |
| v
Rules, schema, Query results and
permissions application services

Interaction: users express needs -> developers build applications
-> DB access is governed by DBA policies -> DBMS stores/retrieves data
-> results return to users.
```

1. Database Administrator (DBA)

The DBA is responsible for the overall operation, security and availability of the database. Main duties include defining or maintaining schemas, creating user accounts and privileges, enforcing security, monitoring performance, managing backups and recovery, ensuring data integrity, planning storage and handling upgrades. The DBA also coordinates with developers to ensure applications access data safely and efficiently.

2. End user

An end user interacts with a database directly or through an application. Examples include a bank customer checking an account balance, a cashier recording a sale, or a university student viewing marks. End users enter, retrieve or update data according to their permissions. They should provide accurate information and protect credentials and confidential data.

3. Application developer

Application developers analyze requirements and build programs that communicate with the DBMS. They design user interfaces, write application logic and SQL queries, validate input, handle errors and test database operations. They should use secure coding practices, parameterized queries and only the access privileges needed by the application.

4. How they interact

First, end users and the organization identify the information and tasks required. Developers translate these requirements into an application and database queries. The DBA defines appropriate structures, permissions, backup plans and performance controls. When a user performs an operation, the application sends a request to the DBMS; the DBMS checks access and executes the request. The result is returned to the application and shown to the user. Developers and the DBA work together to troubleshoot errors and improve performance.

5. Example: university result portal

Students use a portal to view marks. Developers build the login page, query logic and result display. The DBA maintains student and marks data, restricts access so each student sees only authorized records, backs up the database and monitors performance. Faculty may have separate privileges to enter marks. This example shows how responsibilities are different but connected.

Conclusion

The DBA protects and maintains the database, developers create reliable applications, and end users use the resulting services. Clear responsibility, secure access and cooperation among all three roles make the DBMS environment dependable.

Suggested 20-mark coverage: roles and responsibilities (9), concept map (5), interaction example (4), conclusion and clarity (2).